# Efficient Privacy-Preserving High-Dimensional Approximate Nearest Neighbor Search

Lyu Xu*, Guozhong Li†, and Qichen Wang*
*Nanyang Technological University, Singapore
†King Abdullah University of Science and Technology, Thuwal, Saudi Arabia
*{lyu.xu,qichen.wang}@ntu.edu.sg, †guozhong.li@kaust.edu.sa

***Abstract*—In the AI era, approximate nearest neighbor (ANN) search on vector databases is critical as it supports similarity retrieval over high-dimensional vectors, which is essential for many large language model (LLM) applications. To reduce costs and improve scalability, vector databases are often outsourced to cloud servers, raising privacy concerns for users whose query vectors contain sensitive information. In this work, we study the problem of asymmetric privacy-preserving ANN search, where the vector database is public but user queries must remain private from an untrusted cloud server. To solve this problem, we propose ObliSC, an efficient privacy-preserving search framework that protects query privacy against access-pattern leakage. ObliSC consists of (i) a search base that retrieves ANNs with a strong privacy guarantee of full obliviousness; and (ii) parallel optimization strategies that accelerate candidate retrieval and refinement while preserving full obliviousness and retrieval quality. Experiments on six standard ANN benchmarks show that ObliSC not only achieves efficiency and result quality competitive with baseline methods but also maintains a lightweight index compared with the evaluated privacy-preserving approaches.**

***Index Terms*—Approximate nearest neighbor search, query privacy, access pattern protection, obliviousness.**

## I. Introduction

Vector databases support AI applications that retrieve objects by semantic similarity. In retrieval-augmented generation (RAG), for example, a dense retriever embeds a question and searches a passage index for context that a language model can use to generate an answer [1]. Approximate nearest neighbor (ANN) search is a basic operation for such retrieval: it finds vectors close to the query without comparing it with every item in a large collection [2]–[4].

Nowadays, the cloud service is popular since the computational resources of individual users are limited. In this scenario, the query and the collection may have different privacy requirements. Consider a publisher that hosts a vector index of its technical manuals in its own cloud. The publisher already knows the manuals and can inspect their vectors in plaintext. A customer, however, may search the manuals with an embedding of an unpublished incident report. Even revealing which manuals the customer consulted could identify the product or fault under investigation. As there are increasing concerns about disclosing personal information [5], privacy rights and data protection [6], [7], and the design of privacy-preserving services [8], it is critical to protect what users reveal when they interact with a service provider (SP). Hence, in this paper, we study the corresponding *asymmetric* ANN search problem: the database and index are public to SP, whereas the user's query must remain private from SP.

Prior work such as Preco [9] protects the user's query while bounding what the client learns about the database. However, Preco requires the user to query two servers and relies on them not pooling their respective query shares. In contrast, we focus on a single-server design for asymmetric ANN search that does not require a second, non-colluding server.

Table 1 compares representative nearest neighbor search methods in terms of their system models, privacy targets, and performance. PP-ANNS [11] protects query and database contents using encrypted comparisons, but does not establish a formal privacy guarantee for query-dependent access patterns. PACMANN [12] protects queries over a public database by letting the user traverse a graph and privately retrieve information at each hop. Panther [14] protects the user's query from the server and the server's database from the user. BiSON [13] protects both the query and the outsourced database from the storage server and uses disk-compatible oblivious RAM (ORAM) to hide graph accesses. In the federated setting, FedKNN [15] protects the participating providers' data during collaborative $k$NN search, but its threat model does not require the user's query to be hidden from those providers.

Like PACMANN, we study asymmetric ANN search in the single-server setting. This raises a natural question: *can we answer such queries more efficiently while providing a formal guarantee of query privacy against access-pattern leakage?*

**Our Approach.** To answer this question, our idea is to design an *oblivious*[1] search framework that hides query-dependent memory accesses from SP. This goal leads to two challenges. ***Challenge 1:*** *An efficient oblivious search framework.* Efficient ANN search relies on indexes to restrict the search to promising candidates. However, incorporating an index into an oblivious framework requires protecting the entire search process. For example, graph traversal and cluster probing use query-dependent addresses, while candidate aggregation and refinement can reveal which items were selected. Protecting these operations without losing the efficiency of indexed search is challenging. Recent advances in subspace-collision search, including SuCo [4] and TaCo [16], offer an opportunity to design an efficient oblivious search framework. Building on their separation of cell probing, collision-score aggregation,

[1]The notions of obliviousness are formally defined in Section II-B.

Table 1: Summary of representative privacy-preserving ANN approaches. Quality and cost are qualitative summaries based on the respective papers (+: favorable; △: mixed; −: unfavorable).

| Method | System model | Quality | Cost | Access pattern protection | Query protection | Database protection |
|---|---|---|---|---|---|---|
| Plaintext ANN (*e.g.*, SuCo [4]) | Single-server | + | + | None | None | None |
| Path ORAM [10] | Single-server | — | − | Block addresses | — | From SP |
| PP-ANNS [11] | Single-server | + | △ | No trace guarantee | Yes | From SP |
| PACMANN [12] | Single-server | △ | △ | Private graph reads | Yes | Not required (public database) |
| BiSON [13] | Single-server | △ | △ | Oblivious graph reads | Yes | From SP |
| Panther [14] | Single-server | + | △ | Private selection and reads | Yes | From user |
| FedKNN [15] | Multi (federated) | + | △ | SP-side protection | None | Across SPs |
| Preco [9] | Multi (two) | + | △ | Private bucket reads | Yes | Bounded leakage to user |
| ObliSC **(ours)** | Single-server | + | △ | obliviousness | Yes | Not required (public database) |

and refinement, we jointly design ObliSC, a privacy-preserving asymmetric ANN search framework. ObliSC's search base consists of a K-means-based inverted multi-index and fully oblivious (FO) search procedures: the index supports uniform cell retrieval, candidate activation selects promising items within a public probe budget, and refinement uses their collision scores to identify the final results, while hiding query-dependent accesses throughout the search. These stages execute within a trusted execution environment, which protects private data values during computation, thereby combining data confidentiality with access-pattern protection to preserve query privacy throughout the search.

***Challenge 2:*** *Exploiting parallelism while preserving obliviousness.* Parallel execution is a natural direction for further reducing the query latency of ObliSC's FO search base. However, parallelizing oblivious sorting alone does not address the dependencies across search stages: candidate activation is independent across subspaces, whereas collision-score accumulation updates shared state and determines subsequent candidate selection. Directly executing these updates concurrently requires coordination of both the scores and the ORAM state, which can limit parallelism. We therefore separate query-dependent local computation from global aggregation in public item order. Each subspace independently activates candidates and accumulates local collision scores, which are then combined through an oblivious merge to support global selection and refinement. Together with parallel oblivious sorting and distance computation, this design exposes parallelism while keeping worker assignments, access schedules, and synchronization points independent of the query to achieve the guarantee of full obliviousness. It preserves the search base's scoring semantics and retrieval quality at the cost of additional storage and aggregation work.

**Contributions.** Our contributions are as follows:

- We formulate single-server asymmetric ANN search over a public vector database with a private query, and specify full obliviousness under this setting.
- We propose ObliSC, a privacy-preserving asymmetric ANN search framework with a subspace-collision-based search base, whose K-means-based inverted multi-index, candidate activation, and candidate refinement are jointly designed to protect query-dependent access patterns throughout the search. We prove that ObliSC's search base satisfies the privacy guarantee of full obliviousness.
- We develop parallel ObliSC by decomposing the shared collision-score updates during the candidate refinement into independent local accumulation and oblivious global aggregation, enabling concurrent processing across subspaces without contention on a shared score ORAM. Together with parallel candidate activation, oblivious selection, and distance computation, this design exploits parallelism across the online search stages. We prove that it preserves full obliviousness and the search base's results, and analyze the trade-offs among total work, parallel running time, and auxiliary space.
- We implement a prototype of ObliSC and evaluate its retrieval quality, search time, and index construction on six standard ANN benchmarks, with comparisons to non-private and privacy-preserving baselines. The experimental results demonstrate ObliSC's competitive performance.

**Organization**. The rest of this paper is organized as follows. We introduce the background in Section II. Section III presents the overall framework of ObliSC together with ObliSC's FO search base. Then, we present ObliSC's parallel optimization strategies in Section IV. The experimental evaluation is presented in Section V and the related work is discussed in Section VI. We conclude this paper in Section VII.

## II. Background

We first introduce the preliminaries in Section II-A. We then show the system model and security basis in Section II-B, and summarize SuCo's insecure subspace-collision pipeline for high-dimensional ANN search in Section II-C. Some frequently used notations are summarized in Table 2.

### A. Preliminaries

**Vector Similarity Search.** Let $\mathcal{X} = \{X_1, \ldots, X_n\}$ be a database of $n$ vectors, where each vector item $X = (x_1, \ldots, x_d) \in \mathbb{R}^d$ has $d$ coordinates. We also use $\mathcal{X} \in \mathbb{R}^{n \times d}$ for its matrix representation, with row $j$ corresponding to item $X_j$. Given a query vector $q \in \mathbb{R}^d$ in the same metric space, similarity search retrieves database items according to a distance function $\rho$. In this paper, we consider the Euclidean distance function

$$\rho(q, X) = \|q - X\|_2 = \left( \sum_{\ell=1}^{d} (q[\ell] - x_\ell)^2 \right)^{1/2},$$

Table 2: Frequently used notations.

| Notation | Meaning |
|---|---|
| $X = (x_1, \ldots, x_d)$ | a $d$-dimensional database vector |
| $\mathcal{X} \in \mathbb{R}^{n \times d}$ | vector database $\{X_1, \ldots, X_n\}$ |
| $q$ | private query vector in $\mathbb{R}^d$ |
| $\rho(\cdot, \cdot)$ | distance function for vector similarity search |
| $k, E_k(q), A_k(q)$ | query parameter, exact top-$k$ answer set, and approximate top-$k$ answer set |
| $\Pi$, $\mathcal{A}$, $\mathcal{L}$ | protocol, polynomial-time adversary, and common public information |
| $\mathsf{Access}^{\Pi}(q, \mathcal{X})$ | the access pattern of operations that defines $\mathcal{A}$'s observation during the execution of $\Pi$ on $\mathcal{X}$ for $q$ |
| $\hat{\alpha}, \beta$ | collision parameter and re-rank ratio |
| $\alpha$ | collision ratio for SuCo |
| $\mathsf{SC}, \mathsf{SC}_i$ | global subspace-collision score array and local score array for subspace $S_i$, respectively, where entry $j$ stores the score of item $X_j$ |
| $R_i$, $\mathcal{R}$ | activated identifier multiset for $S_i$, collection of these multisets |
| $\mathcal{S}, N_s$ | public subspace schedule and number of subspaces |
| $S_i, P_i, S_i^{(r)}, P_i^{(r)}$ | subspace, projection function, and half-subspace/half-projection used by IMI |
| $\sqrt{\kappa}, \kappa, t$ | number of K-means clusters, IMI cell count per subspace, and K-means iteration count |
| $C_i^{(r)}, a_i^{(r)}, q_i^{(r)}$ | centroid set, assignment array, and projected query in half $r \in \{1, 2\}$ of $S_i$ |
| $a_i^{(r)}[j]$ | assignment of the $j^{\text{th}}$ item $X_j$ in $\mathcal{X}$ *w.r.t* the $r^{\text{th}}$ ($r = 1, 2$) half dimensions in subspace $S_i$ |
| $Max[i], \perp$ | the maximum cell size of the inverted multi-index $IMI_i$ for subspace $S_i$, a unique virtual item identifier |
| $IMI_i, IMIs$ | inverted multi-index for $S_i$, list of these indexes |
| $p, \ell_i, \ell$ | number of workers per subspace, padded activation size $\ell_i = \hat{\alpha} \cdot Max[i]$, and total activation size $\ell = \sum_i^{N_s} \ell_i$ |

where $q[\ell]$ is the $\ell$-th query coordinate. Then, given $1 \le k \le n$, *exact* top-$k$ search returns a set $E_k(q)$ of $k$ vectors in $\mathcal{X}$ nearest to $q$. That is,

$$E_k(q) = \arg \min_{\mathcal{X}' \subseteq \mathcal{X},\ |\mathcal{X}'|=k} \sum_{X_i \in \mathcal{X}'} \rho(q, X_i).$$

**Approximate Top-$k$ Nearest Neighbor (A$k$NN) Search.** To enable efficient retrieval, *approximate top-k nearest-neighbor* search returns a set $A_k(q)$ of $k$ database vectors, aiming for high overlap with $E_k(q)$ while avoiding an exhaustive search for the exact answers. Following existing A$k$NN works [4], [11], [12], we adopt the metric of recall to evaluate the utility of an approximate search result $A_k(q)$. That is,

$$\text{Recall@}\, k(q) = \frac{\left|A_k(q) \cap E_k(q)\right|}{k}. \tag{1}$$

Then, we introduce the notions of subspace and projection used for subspace collision-based A$k$NN [4], [16].

**Subspaces and Projections.** Let $\mathcal{S} = \{S_1, \ldots, S_{N_s}\}$ partition $d$ coordinate dimensions into $N_s$ nonempty, disjoint groups. The projection $P_i$ selects the coordinates in $S_i$, giving

$$q_i = P_i(q), \qquad \mathcal{X}_i = \{P_i(X_j) : 1 \le j \le n\}.$$

For the $N_s$ disjoint subspaces, squared Euclidean distance satisfies

$$\|q - X_j\|_2^2 = \sum_{i=1}^{N_s} \|q_i - P_i(X_j)\|_2^2.$$

Projected distance rankings may differ across subspaces. The subspace-collision approach in Section II-C therefore efficiently aggregates rank evidence from multiple projections via distance-based indexes in subspaces to select candidates, and uses the original-space distance for final ranking.

### *B. System Model and Security Basis*

**System Model.** We consider a cloud computing scenario with query privacy protection: the user submits a private query vector to a service provider (SP) for A$k$NN retrieval over a public vector database and its query-independent index. We adopt the widely used assumption in cloud computing that SP is an *honest-but-curious* adversary: it follows the computational protocol but may infer sensitive information about the user's private query. Under this model, we consider the following two kinds of privacy as the *privacy target*:

- **Data privacy.** The data privacy is to protect the user's query vector and any query-dependent intermediate values except for the query parameter $k$; and
- **Access pattern privacy.** The access pattern privacy is to ensure that SP cannot infer any information that may compromise the data privacy.

Next, we introduce the security basis used in this paper.

**Security Basis.** To protect data privacy, there have been cryptographic techniques supporting secure computational operators, such as secret sharing [17], [18], homomorphic encryption scheme [19]–[21], and trusted execution environment (TEE) [22]–[24]. To achieve the above privacy target, we can leverage these techniques to preserve the data privacy and mainly focus on designing secure search protocols with protection on the *access pattern privacy*.

To protect access pattern privacy, obliviousness is a standard security notion that provides formal privacy guarantee. We adopt full obliviousness [25] to protect the complete observable search execution pattern.

**Definition 1 (Full Obliviousness).** *Given a public vector database $\mathcal{X}$, its index, and the public algorithm parameters, including $d$ and $k$, denote this common public information by $\mathcal{L}$. Then, for any two query vectors $q, q' \in \mathbb{R}^d$ with the same public parameters, a protocol $\Pi$ is* fully oblivious (or simply FO) *if, for every probabilistic polynomial-time adversary $\mathcal{A}$, there exists a negligible function $\delta(\lambda)$ such that*

$$\begin{aligned} &\Pr\left[\mathcal{A}\big(\mathcal{L}, \mathsf{Access}^{\Pi}(q, \mathcal{X})\big) = 1\right] \\ &\quad \le \Pr\left[\mathcal{A}\big(\mathcal{L}, \mathsf{Access}^{\Pi}(q', \mathcal{X})\big) = 1\right] + \delta(\lambda). \end{aligned} \tag{2}$$

*where (i) $\mathsf{Access}^{\Pi}(q, \mathcal{X})$ denotes the observable access trace of executing $\Pi$ on $q$ and $\mathcal{X}$; (ii) $\mathcal{A}$ outputs a bit to distinguish the two trace distributions. The probabilities are over the randomness of $\Pi$ and $\mathcal{A}$; and (iii) $\lambda$ is the cryptographic security parameter, and $\delta(\lambda)$ is a negligible upper bound on the adversary's distinguishing advantage between the observable access traces induced by $q$ and $q'$ under the same public parameters.*

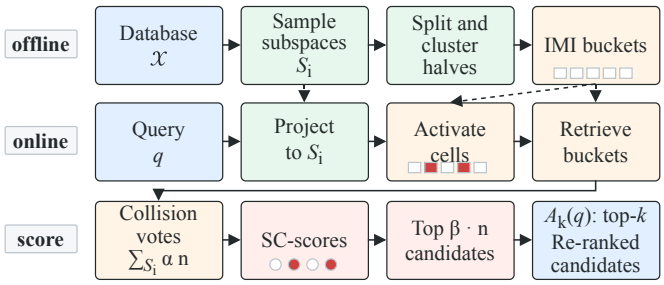



Figure 1: Plaintext SuCo pipeline from IMI construction to top-$k$ reranking.

Now, we are ready to present the problem statement.

**Problem Statement.** Given the database $\mathcal{X}$ and a private user query $q$ with parameter $k$, the goal is to design secure A$k$NN search approaches with access pattern privacy preservation under the system model above, while the data privacy can be protected via standard cryptographic techniques, *e.g.*, TEE.

### C. Pipeline for Subspace Collision-Based A$k$NN Search

Figure 1 summarizes SuCo's plaintext pipeline [4] that consists of offline index construction followed by online search.

- **Offline index construction.** SuCo partitions the $d$ dimensions into $N_s$ sampled subspaces and builds an inverted multi-index (IMI) for each subspace $S_i$. It then splits $S_i$ into two halves and clusters each half into $\sqrt{\kappa}$ centroids using K-means. Each identifier is stored in one of $\kappa$ IMI cells according to its nearest centroid pair.
- **Online search.** SuCo projects the user query $q$ into each subspace and computes its distances to both centroid sets. Then, it prioritizes IMI cells by the sums of these distances and retrieves identifiers until at least $\alpha \cdot n$ items are activated as candidates, where $\alpha$ is the collision ratio and $n$ is the database size. Next, SuCo (i) counts one collision vote for each retrieved item per subspace, and (ii) retains the $\beta \cdot n$ highest-scoring candidates, where $\beta$ is the re-rank ratio. Finally, SuCo re-ranks these $\beta \cdot n$ candidates by original-space distances to return the closest $k$ ones as $A_k(q)$.

SuCo's index cell retrieval, candidate activation, and refinement accesses depend on the user query $q$ and expose its access pattern. In Section III, we present our ObliSC search framework that protects $q$ against access-pattern leakage.

## III. ObliSC's Overall Search Framework

This section illustrates the overview and the fully oblivious search base of our subspace collision-based ObliSC search framework. Due to space limitations, some detailed proofs are presented in Appendix A of the technical report [26].

### A. ObliSC's Overview

ObliSC has three components: (i) one-time offline *query-independent* inverted multi-index (IMI) construction, (ii) fully oblivious (FO) IMI based candidate activation, and (iii) FO top-$k$ refinement. Figure 2 shows the query user and the cloud server that runs ObliSC's overall framework, while Algorithm 1 gives the corresponding procedure.

**Algorithm 1:** ObliSC's overall framework

**Input:** database $\mathcal{X}$ with $n$ $d$-dimensional items, query $q$ with parameter $k$, subspace number $N_s$, collision parameter $\hat{\alpha}$, and re-rank ratio $\beta$
**Output:** approximate top-$k$ answer set $A_k(q)$

```
// (i) Offline Query-Independent Index Construction
1 IndexConstruct (·)        // generate IMI index
// Online Search Process
2 initialize SC[j] ← 0 for each item X_j, j ∈ [1, n] in X
3 divide the d dimensions into N_s subspaces S_1, ..., S_{N_s} and
    project q into every subspace
4 initialize multiset R ← ∅ of elements in activated cluster
    sets containing answers
5 for i ← 1 to N_s do
      // (ii) FO Candidate Activation
6     R ← R ∪ {CanActive (·)}
// (iii) FO Top-k Refinement: count the SC and refine the top-k answers
7 A_k(q) ← TopkRefine (·)
8 return A_k(q)
```

Taking as inputs the database $\mathcal{X}$ with $n$ $d$-dimensional vector items, a query $q$ with parameter $k$, the subspace number $N_s$, the collision parameter $\hat{\alpha}$, and the re-rank ratio $\beta$, Algorithm 1 outputs an approximate top-$k$ answer set $A_k(q)$. First, as ObliSC's component (i), Line 1 invokes the `IndexConstruct` procedure to generate the one-time offline subspace collision-based IMI index. Then, Lines 2-4 initialize the online query process for the input query $q$ by setting the *subspace collision* (SC) score SC of each data item as 0, and dividing and projecting $q$ into each of the $N_s$ subspaces via projections. For each subspace, Lines 5-6 invoke the `CanActive` procedure as ObliSC's component (ii) to obtain the activated candidate item (identifier) set by using IMI in an FO manner to find promising items. The elements in these sets are inserted into multiset $\mathcal{R}$. Line 7 invokes the FO `TopkRefine` procedure as ObliSC's component (iii) to refine the top-ranked candidates in $\mathcal{R}$ and obtain an A$k$NN answer set $A_k(q)$, which is returned as the search result in Line 8. The following subsections will illustrate each of these three components.

### B. SC-based Index Construction

According to the existing SC-based A$k$NN search methods [4], [27], the IMI construction depends on the database $\mathcal{X}$ only, which is independent of the input query. However, these methods do not consider the fixed-shape index design, which is critical for achieving FO online search since query-dependent online search processing may access index entries of different sizes, compromising access-pattern privacy. Therefore, we propose to construct fixed-shape IMI to facilitate the oblivious design for ObliSC's online search. In Figure 2, the rectangle on the right illustrates ObliSC's IMI layout for a single subspace and the padded cell representation.

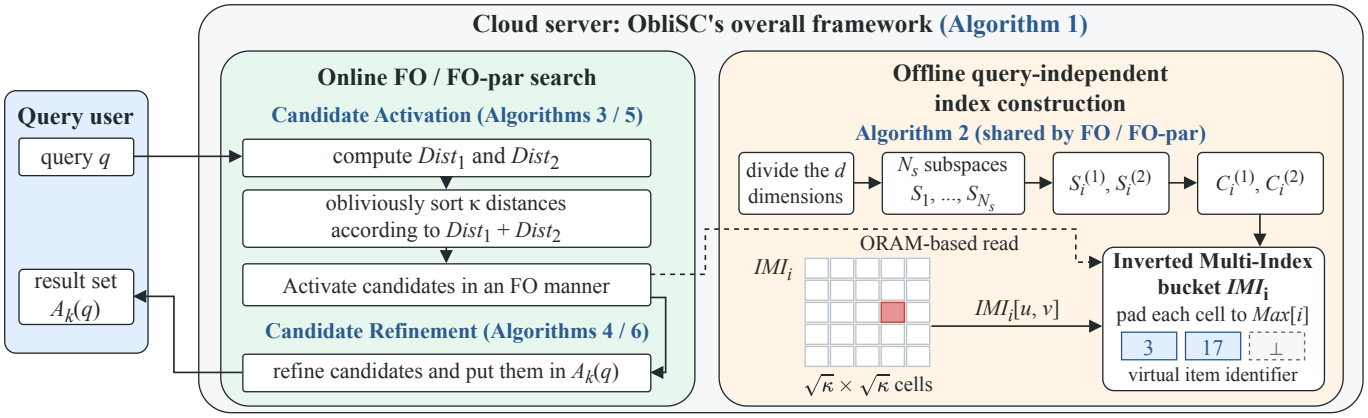



Figure 2: ObliSC system model: offline query-independent index construction and online FO search processing.

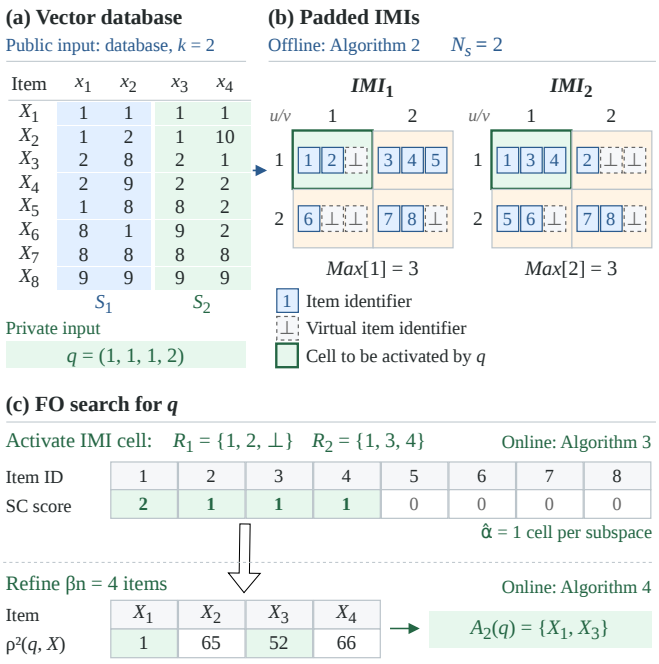



Figure 3: A running example of ObliSC.

**Algorithm 2:** The `IndexConstruct`$(\cdot)$ procedure

**Input:** database $\mathcal{X}$ with $n$ $d$-dimensional items, subspace number $N_s$, IMI cell count per subspace $\kappa$, and K-means iterations $t$

**Output:** centroid list $centroids$, IMI list $IMIs$, and maximum cluster size array $Max$

```
// Initialization
1  initialize centroids with length 2N_s, IMIs with length N_s
2  divide the d dimensions into N_s subspaces S_1, ..., S_{N_s}
3  divide all items into subspace datasets X_1, ..., X_{N_s}
// KMeans-Based Index Construction
4  for i ← 1 to N_s do
5      initialize a map IMI_i as the inverted multi-index
6      split S_i into S_i^(1) and S_i^(2), split X_i into X_i^(1) and X_i^(2)
7      (C_i^(1), a_i^(1)) ← KMeans(X_i^(1), √κ, t)
8      (C_i^(2), a_i^(2)) ← KMeans(X_i^(2), √κ, t)
9      append C_i^(1) and C_i^(2) to centroids
10     for j ← 1 to n do
11         u ← a_i^(1)[j], v ← a_i^(2)[j]
12         IMI_i[u, v].append(j)        // IMI_i: √κ × √κ cells
13     Max[i] ← the maximum cell size of IMI_i
14     pad all cells in IMI_i to size Max[i] using a unique
         virtual item identifier ⊥
15     append IMI_i to IMIs
16 return ⟨centroids, IMIs, Max⟩
```

The IMI construction is shown in Algorithm 2. Taking as inputs the database $\mathcal{X}$ with $n$ $d$-dimensional items, the subspace number $N_s$, the number of K-means clusters $\sqrt{\kappa}$ (or the IMI cell count per subspace $\kappa$), and the K-means iterations $t$, the `IndexConstruct` procedure outputs as IMI the centroid list $centroids$, the IMI list $IMI$s, and an array $Max$ of each subspace's maximum cluster size used for FO design. First, Line 1 initializes *centroids* with length $2 \cdot N_s$ and *IMI*s with length $N_s$, and Line 2 divides the $d$ dimensions into $N_s$ subspaces via *uniformly sampling without replacement*. Accordingly, Line 3 divides all items into $N_s$ subspace datasets $\mathcal{X}_1, \ldots, \mathcal{X}_{N_s}$. Then, Lines 4-14 construct the IMI $IMI_i$ for the $i^{\text{th}}$ $(1 \le i \le N_s)$ subspace $S_i$ as follows: (i) Line 5 initializes a 2-dimensional map as the initial $IMI_i$; (ii) Line 6 splits $S_i$ into two disjoint and lower-dimensional halves $S_i^{(1)}$ and $S_i^{(2)}$ *w.r.t* the first $\lfloor d_i/2 \rfloor$ and the remaining $d_i - \lfloor d_i/2 \rfloor$ dimensions, where $d_i = \dim(S_i) = d/N_s$, respectively. Similarly, each subspace dataset $\mathcal{X}_i$ is also split into two halves $\mathcal{X}_i^{(1)}$ and $\mathcal{X}_i^{(2)}$; (iii) for the first (*resp.* the second) half dimensions of $S_i$, Line 7 (*resp.* Line 8) invokes *K-means* on $\mathcal{X}_i^{(1)}$ (*resp.* $\mathcal{X}_i^{(2)}$) to compute the centroid set $C_i^{(1)}$ (*resp.* $C_i^{(2)}$) of $\sqrt{\kappa}$ centroids in $S_i$ together with the assignment array $a_i^{(1)}$ (*resp.* $a_i^{(2)}$) of $n$ assignments (*i.e.*, the *nearest* centroid to each of the $n$ items). Line 9 appends $C_i^{(1)}$ and $C_i^{(2)}$ in *centroids*; (iv) Lines 10-12 append identifier $j$ to the 2-dimensional $IMI_i$ with index key pair $\left(a_i^{(1)}[j], a_i^{(2)}[j]\right)$ for the $j^{\text{th}}$ $(1 \le j \le n)$ item; and (v) for the purpose of FO design, Line 13 stores in $Max[i]$ the maximum cell size of $IMI_i$, and then Line 14 pads each cell in $IMI_i$ to size $Max[i]$ using a unique virtual item identifier $\perp$, which indicates a virtual item having $+\infty$ distances from any queries. Finally, Line 16 returns $\langle centroids, IMIs, Max \rangle$.

**Algorithm 3:** FO `CanActive(·)` procedure

**Input:** Subspace $S_i$, projected query $q_i$, collision parameter $\hat{\alpha}$, dataset size $n$, IMI cell count per subspace $\kappa$, and $\langle centroids, IMIs, Max \rangle$

**Output:** Multiset $R$ of identifiers for activated items whose item points collide with $q_i$ in $S_i$

```
// Initialization for q_i
```
1 compute array $Dist_1$ (*resp.* $Dist_2$) of distances between $q_i^{(1)}$ (*resp.* $q_i^{(2)}$) and each of the $\sqrt{\kappa}$ centroids in subspace $S_i^{(1)}$ (*resp.* $S_i^{(2)}$)

2 invoke `Obli_Sort` to sort all $\kappa$ pairwise distance sums $Dist_1[u] + Dist_2[v]$ in ascending order and store the corresponding centroid identifier pairs $(u, v)$ in array $ID$ in the same order

```
// Candidate Activation
```
3 $R \leftarrow \emptyset$

4 **for** $pos \leftarrow 1$ **to** $\hat{\alpha}$ **do**

5 $\quad u, v \leftarrow ID[pos]$

6 $\quad cluster \leftarrow IMI_i[u, v]$ `// ORAM-based read`

7 $\quad R \leftarrow R \uplus cluster$ `// ⊎: multiset union`

8 **return** $R$

**Example 1** (IMI Index Construction)**.** *Consider a running example of a database that consists of the eight four-dimensional vectors in Figure 3(a), with a private query $q = (1, 1, 1, 2)$ and $k = 2$. Let $N_s = 2$ with coordinate groups $S_1 = \{1, 2\}$ and $S_2 = \{3, 4\}$, let $\kappa = 4$, giving two centroids per half-subspace, and let the K-means iterations $t = 2$.*

*For each one-dimensional half-subspace, the IMI construction invokes K-means for 2 iterations. The resulting assignments give the IMIs in Figure 3(b). For instance, $IMI_1[1, 1]$ contains identifiers $1, 2$, whereas $IMI_1[1, 2]$ contains $3, 4, 5$. Both IMIs have maximum cell size three, so $Max[1] = Max[2] = 3$. Padding changes $IMI_1[1, 1]$'s cell to $(1, 2, \perp)$.*

**Analysis.** $centroids$, $IMIs$, and $Max$ provide efficient search structure used by the later candidate activation procedure, and their constructions depend on database $\mathcal{X}$ and public parameters only. That is, the input query $q$ has no effect on the access pattern $\mathsf{Access}^{\Pi}(q, \mathcal{X})$ of `IndexConstruct`. According to Definition 1, we can easily know that `IndexConstruct` satisfies full obliviousness.

### C. FO *Candidate Activation*

The existing activation procedure dynamically activates the candidate clusters that contain the query answers, which involves a lot of query-dependent code branches and hence fails to satisfy full obliviousness. To handle this, we propose an FO candidate activation procedure.

As shown in Algorithm 3, taking as inputs the subspace $S_i$, the projected query $q_i$ in $S_i$, the collision parameter $\hat{\alpha}$, the dataset size $n$, the IMI budget $\kappa$, and ObliSC's index structure $\langle centroids, IMIs, Max \rangle$, the `CanActive` procedure outputs the activated candidate (item identifier) multiset $R$ for $S_i$. First, Line 1 computes and stores in arrays $Dist_1$ and $Dist_2$ the distances from the projected queries $q_i^{(1)}$ and $q_i^{(2)}$ (*i.e.*, the projected queries of $q_i$ *w.r.t* the first and second half dimensions in $S_i$) to the two centroid groups $C_i^{(1)}$ and $C_i^{(2)}$, respectively. Then, Line 2 invokes the data-oblivious sorting procedure `Obli_Sort` to sort all $\kappa$ pairwise distance sums in ascending order and stores the corresponding centroid identifier pairs in array $ID$ in the same order. To activate candidate clusters, the existing subspace-collision pipeline uses a collision ratio $\alpha$ and dynamically retrieves at least $\alpha \cdot n$ items from the activated clusters. Different from such a query-dependent dynamic activation, `CanActive` adopts an input collision parameter $\hat{\alpha}$ as a fixed number of $IMI_i$ cell accesses. This ensures that Lines 4-7 always retrieve $Max[i]$ identifiers from each of the $\hat{\alpha}$ cells corresponding to the first $\hat{\alpha}$ centroid identifier pairs in $ID$. It is worth noting that Line 6 adopts a standard ORAM-based memory access approach (*e.g.*, Path-ORAM [10] and Circuit ORAM [28]) to read the $Max[i]$ items stored in a specific cell of IMI bucket $IMI_i$, so as to protect the memory access pattern. Finally, Line 8 returns multiset $R$ of $\hat{\alpha} \cdot Max[i]$ activated item identifiers.

**Example 2** (FO Candidate Activation)**.** *Following Example 1, we consider the candidate activation for query $q = (1, 1, 1, 2)$. Assume that $\hat{\alpha} = 1$. For $q$, the centroid pair with the smallest distance sum is $(1, 1)$ in both IMIs. Therefore, as shown in Figures 3(b)-3(c),* `CanActive` *reads $IMI_1[1, 1]$ and $IMI_2[1, 1]$ in an ORAM-based method to activate the candidates with identifiers in $R_1 = \{1, 2, \perp\}$ and $R_2 = \{1, 3, 4\}$, respectively.*

**Analysis.** Throughout our complexity analysis, we treat ORAM as a black box and assume that accessing one of $N$ blocks, each containing $b$ scalar entries, takes $O(b \cdot \log N)$ time. Each padded IMI cell is one block of $Max[i]$ identifiers, each score is a constant-size block, and each database item is a block of $d$ coordinates. Then, for the time complexity, `CanActive` takes $O(\sqrt{\kappa} \cdot d/N_s)$ time to compute $Dist_1$ and $Dist_2$ (Line 1),[2] $O(\kappa \log^2 \kappa)$ time to obliviously sort $\kappa$ distances (Line 2), and $O\big(\hat{\alpha} \cdot Max[i] \cdot \log \kappa^2\big)$ time to obtain the activated candidate set $R$ (Lines 3-7). Hence, the total time complexity of invoking $N_s$ times of `CanActive` is $O\big(\sqrt{\kappa} \cdot d + N_s \cdot \kappa \log^2 \kappa + \hat{\alpha} \cdot \log \kappa \cdot \sum_{i=1}^{N_s} Max[i]\big)$.

Our privacy analysis assumes protected private values and oblivious execution under fixed public inputs and parameters.[3] An indistinguishability analysis of Algorithm 3 under these assumptions yields the following lemma.

**Lemma 1.** `CanActive` *satisfies full obliviousness.*

*Proof.* Consider two inputs of `CanActive` that have the same public index and parameters but different query data values. Comparing Algorithm 3 for both inputs, we have the following: (i) Lines 1-2 compute a fixed number of distances and obliviously sort all $\kappa$ distance sums, so their access schedules are independent of the query values; and (ii) Lines 3-7 scan the first $\hat{\alpha}$ entries of $ID$ in the same order and retrieve $Max[i]$ identifiers per IMI cell. Although the selected cell

[2] Assume that arithmetic operations take $O(1)$ time.

[3] The TEE protects query values, intermediate values, and results, while query and result transfers remain confidential. We assume adaptively secure ORAM and oblivious local operations, including observable ORAM-internal accesses, with storage and per-access work padded to public bounds.

**Algorithm 4:** FO `Top`$k$`Refine` procedure

**Input:** Subspace number $N_s$, multiset $\mathcal{R}$ of activated candidate item identifier in multisets $R_i$ for all subspaces $S_i$, dataset size $n$, query $q$ with parameter $k$, and re-rank ratio $\beta$
**Output:** Approximate top-$k$ answer set $A_k(q)$

1 **for** $i \leftarrow 1$ **to** $N_s$ **do**
2  **foreach** $point_id \in R_i$ **do**
3  $\mathsf{SC}[point_id]++$  `// ORAM-based update`
4 invoke `Obli_Sort` to sort SC in descending order
5 $A \leftarrow \emptyset$
6 **for** $z \leftarrow 1$ **to** $\beta \cdot n$ **do**
7  let $\tau$ be the item identifier for the $z^{\text{th}}$ element in SC
8  $X' \leftarrow X_\tau$  `// ORAM-based read`
9  append $\langle \rho(X', q), \tau \rangle$ to the next public slot of $A$
  `// compute original distance`
10 invoke `Obli_Sort` to sort $A$ in ascending order of distance
11 retrieve the items of the first $k$ elements in sorted $A$ through ORAM and write them into consecutive slots of $A_k(q)$
12 **return** $A_k(q)$

addresses may differ, every retrieval uses ORAM and the retrieved identifiers are appended to $R$ in a fixed-length scan. Hence, both executions issue the same number of fixed-size ORAM requests. By adaptive ORAM security, the corresponding observable traces are computationally indistinguishable. The remaining access schedules are identical, and both outputs have size $\hat{\alpha} \cdot Max[i]$. Therefore, Equation (2) holds, and Lemma 1 follows. □

### D. FO *top-k Candidate Refinement*

After `CanActive`, ObliSC aggregates the collision votes (*i.e.*, collided candidates) across subspaces and refines a fixed number of promising candidates in the original space.

As shown in Algorithm 4, taking as inputs the subspace number $N_s$, the multiset $\mathcal{R}$ of activated candidate (item) identifiers in multisets $R_1, ..., R_{N_s}$, the dataset size $n$, the query $q$ with parameter $k$, and the re-rank ratio $\beta$, the `Top`$k$`Refine` procedure outputs the approximate top-$k$ answer set $A_k(q)$. First, Lines 1-3 aggregate subspace-collision votes by (i) linearly scanning the activated candidate identifiers in each $R_i$ ($1 \leq i \leq N_s$) and (ii) incrementally increasing the SC of these identifiers through ORAM-protected read-modify-write operations to protect the memory access pattern. Note that each virtual identifier also triggers an ORAM update to a reserved dummy score slot, which is excluded from the subsequent sorting of real-item scores. Then, Line 4 invokes `Obli_Sort` to obliviously sort SC in descending order. The items having the first $\beta \cdot n$ identifiers in the sorted SC are considered as the promising ones to be refined: for each of these $\beta \cdot n$ candidates (Line 6), Line 7 obtains the selected identifier $\tau$, Line 8 retrieves the data of $X_\tau$ through ORAM-based reads, and Line 9 computes the original-space distance $\rho(X_\tau, q)$ and appends the distance–identifier pair to array $A$. Then, to refine these $\beta \cdot n$ candidates, Line 10 obliviously sorts the element pairs in $A$ by their distances in ascending order, and Line 11 retrieves the vectors in the first $k$ elements in sorted $A$ and inserts them into the answer set $A_k(q)$. Finally, Line 12 returns $A_k(q)$ as the output.

**Example 3** (FO Candidate Refinement)**.** *Aggregating $R_1$ and $R_2$ gives real-item scores $(2, 1, 1, 1, 0, 0, 0, 0)$. Virtual entries consume protected dummy operations without contributing to real-item scores, and have distance $+\infty$. The four highest-scoring items are $X_1, X_2, X_3, X_4$; their squared Euclidean distances to $q$ are $1, 65, 52, 66$, respectively. Refinement therefore returns $A_2(q) = \{X_1, X_3\}$. This illustrates why collision scores select candidates but do not determine the final distance ranking. The four-item refinement count remains fixed for $q'$, while the selected identifiers and distances remain private.*

**Analysis.** Let $\ell = \sum_{i=1}^{N_s} \hat{\alpha} \cdot Max[i]$. `Top`$k$`Refine` takes $O(\ell \cdot \log n)$ time to scan the activated items for oblivious write on SC (Lines 1-3), $O(n \cdot \log^2 n)$ time to obliviously sort SC (Line 4) and the elements in $A$ (Line 10), and $O\big(\beta \cdot n \cdot d \cdot \log n\big)$ time to obliviously read $\beta \cdot n$ data items and compute their corresponding original-space distances (Lines 6-9). The vector data retrieval in Line 11 takes $O(k \cdot d \cdot \log n)$ time. Therefore, the total time complexity of `Top`$k$`Refine` is $O\big(\ell \cdot \log n + n \cdot \log^2 n + \beta \cdot n \cdot d \cdot \log n\big)$.

An indistinguishability analysis of Algorithm 4 also establishes Lemma 2 for `Top`$k$`Refine`'s access pattern.

**Lemma 2.** `Top`$k$`Refine` *satisfies full obliviousness.*

Lemma 2 can be proved using an indistinguishability analysis similar to that for Lemma 1. Consequently, the analysis in Section III-B and Lemmas 1-2 establish full obliviousness for all three procedures of ObliSC, which covers `IndexConstruct`, `CanActive`, and `Top`$k$`Refine`. As the access patterns of the operations in Lines 2-4 of Algorithm 1 are also independent of the query values, Theorem 1 follows.

**Theorem 1.** ObliSC *satisfies full obliviousness.*

## IV. Parallel Optimization

In this section, we further improve the execution of ObliSC's search base through parallelism on its candidate activation and refinement, while preserving full obliviousness.

### A. Parallel Candidate Activation

As the $d$ dimensions are divided into $N_s$ subspaces via uniform sampling *without* replacement, each subspace contains a disjoint subset of dimensions. Given the index and projected query, candidate activation in each subspace does not depend on the results from other subspaces. Accordingly, we can directly parallelize candidate activation across subspaces.

First, we allocate a public number $p \geq 1$ of workers to each subspace, giving $w = N_s \cdot p$ workers in total. For example, $N_s = 8$ and $p = 10$ give 80 workers. For each subspace, centroid indices are partitioned into public ranges among the $p$ workers for distance computation. These workers also form all $\kappa$ pairwise distance sums and execute parallel `Obli_Sort` using the standard Batcher's bitonic sorting network [29]. After the FO candidate activation in Lines 5-8, Line 9 invokes **sync** to wait for all tasks for different subspaces to

**Algorithm 5:** parallel FO candidate activation

**Input:** Inputs of Algorithm 3 for all $N_s$ subspaces, and public worker count $p$ per subspace
**Output:** Collection $\mathcal{R} = \{R_1, \ldots, R_{N_s}\}$ of activated identifier multisets

1 assign a fixed group of $p$ workers to each subspace $S_i$, initialize $R_i \leftarrow \emptyset$
2 **for** $i \leftarrow 1$ **to** $N_s$ in parallel **do**
3  compute $Dist_1$ and $Dist_2$ in parallel using the $p$ workers assigned to $S_i$
4  invoke parallel `Obli_Sort` using these $p$ workers to sort the $\kappa$ distance sums in ascending order and store the corresponding centroid pairs in $ID$ in the same order
5  **for** $pos \leftarrow 1$ **to** $\hat{\alpha}$ **do**
6  $u, v \leftarrow ID[pos]$
7  $cluster \leftarrow IMI_i[u, v]$ `// ORAM-based read`
8  $R_i \leftarrow R_i \uplus cluster$ `// ⊎: multiset union`
9 **sync**
10 **return** $\mathcal{R}$

**Algorithm 6:** Parallel FO candidate refinement

**Input:** Inputs of Algorithm 4, and public worker count $p$
**Output:** Approximate top-$k$ answer set $A_k(q)$

1 **for** $i \leftarrow 1$ **to** $N_s$ in parallel **do**
2  **for** $j \leftarrow 0$ **to** $n$ **do**
3  $\mathsf{SC}_i[j] \leftarrow 0$ `// ORAM-based write`
4  **for** $z \leftarrow 1$ **to** $\ell_i = \hat{\alpha} \cdot Max[i]$ **do**
5  $a \leftarrow$ `Obli_Set`$(R_i[z], \perp)$
6  $\mathsf{SC}_i[a] + +$ `// ORAM-based update`
7  **for** $j \leftarrow 1$ **to** $n$ **do**
8  $V_i[j] \leftarrow \mathsf{SC}_i[j]$ `// ORAM-based read`
9 **sync**
10 **for** $j \leftarrow 1$ **to** $n$ in parallel **do**
11  $\mathsf{SC}[j] \leftarrow \sum_{i=1}^{N_s} V_i[j]$
12 **sync**
13 invoke parallel `Obli_Sort` using $p$ workers to sort $\mathsf{SC}$ in descending order
14 **for** $z \leftarrow 1$ **to** $\beta \cdot n$ **do**
15  let $\tau$ be the item identifier for the $z^{\text{th}}$ element in $\mathsf{SC}$
16  $W[z] \leftarrow \langle X_\tau, \tau \rangle$ `// ORAM-based read`
17 **for** $z \leftarrow 1$ **to** $\beta \cdot n$ in parallel **do**
18  $A[z] \leftarrow \langle \rho(W[z].X, q), W[z].\tau \rangle$ `// use p workers to compute original distances`
19 **sync**
20 invoke parallel `Obli_Sort` using $p$ workers to sort the elements in $A$ in ascending order based on their distances
21 retrieve the items of the first $k$ elements in sorted $A$ through ORAM and write them into consecutive slots of $A_k(q)$
22 **return** $A_k(q)$

**Operation** `Obli_Set`$(x, \perp)$:
23 **return** $x \cdot \big(1 -$ `Obli_Equal`$(x, \perp)\big)$
`// Obli_Equal returns 1 iff its inputs are equal, and 0 otherwise, without data-dependent accesses or branches`

complete before proceeding. Finally, Line 10 returns collection $\mathcal{R} = \{R_1, \ldots, R_{N_s}\}$.

### *B. Parallel Candidate Refinement*

To parallelize ObliSC's candidate refinement, the main challenge is collision-score aggregation (Lines 1-3 of Algorithm 4) where updates from different subspaces depend on the query and share mutable score state. To preserve the full obliviousness, directly executing these updates concurrently requires coordination of both the scores and the ORAM state, which can serialize execution. Hence, in Algorithm 6, we exploit the additive structure of collision scores to separate independent local accumulation from global aggregation in public item order. This decomposition removes cross-subspace contention during accumulation while preserving the original scoring rule.

**Local Accumulation and Oblivious Aggregation.** For each subspace $S_i$, Lines 1-6 maintain a local score array $\mathsf{SC}_i$ in an independent ORAM, with $n$ real-item slots and a reserved dummy slot at address 0. Each subspace scans all $\ell_i = \hat{\alpha} \cdot Max[i]$ positions of its activated candidate multiset $R_i$ and performs one ORAM-protected read-modify-write update per identifier. The operation `Obli_Set` (Line 23) maps virtual identifiers to the dummy slot while preserving real identifiers, without data-dependent branching. Then, score accumulation proceeds concurrently across subspaces in Line 6, while accesses within each ORAM remain sequential. After the previous local accumulation, each subspace reads its $n$ real-item scores through ORAM in public identifier order into array $V_i$, excluding the dummy slot. Following synchronization, workers independently aggregate the scores in all subspaces according to $\mathsf{SC}[j] = \sum_{i=1}^{N_s} V_i[j], 1 \le j \le n$.

**Parallel Selection and Distance Refinement.** Similar to Line 4 of Algorithm 4, Line 13 of Algorithm 6 uses $p$ workers and the parallel bitonic sorting procedure to rank the global scores. Lines 14-16 retrieve the first $\beta \cdot n$ items sequentially through the shared database ORAM into fixed buffer positions in array $W$. The retrieved items' original-space distances to the query are then computed in parallel using $p$ workers in Lines 17-18, stored as distance–identifier pairs in array $A$. Line 20 invokes parallel oblivious distance sorting on $A$ by the element distances in ascending order, and Line 21 retrieves the vectors identified by the first $k$ records in $A$ through ORAM and writes them into consecutive output slots. Finally, Line 22 returns $A_k(q)$.

### *C. Analysis of Parallel* ObliSC

**Complexity.** We distinguish total work from parallel running time and use the black-box ORAM cost model of Section III, accounting for the size of each accessed block. Let $\ell = \sum_{i=1}^{N_s} \ell_i$ and let $d_i$ denote the dimension of $S_i$ such that $\sum_i d_i = d$. For Algorithm 5, the total work remains the same as that of sequential activation over all subspaces:

$$O\big(\sqrt{\kappa} \cdot d + N_s \cdot \kappa \cdot \log^2 \kappa + \ell \cdot \log \kappa\big).$$

A bitonic network on $n$ records has $O(n \cdot \log^2 n)$ work and $O(\log^2 n)$ depth. With $p$ workers per subspace, the time complexity of parallel activation is therefore

$$O\left(\max_i \left\{ \left\lceil \frac{\sqrt{\kappa}}{p} \right\rceil \cdot d_i + \frac{\kappa \cdot \log^2 \kappa}{p} + \log^2 \kappa + \ell_i \cdot \log \kappa \right\}\right).$$

As the last term includes sequential cell retrieval and copying within each subspace over its individual ORAM, these operations are not divided among its $p$ workers.

For Algorithm 6, its local accumulation (Lines 4-6) performs the same $\ell$ score updates as Algorithm 4, requiring $O(\ell \cdot \log n)$ work. Initializing and reading back the local score ORAMs (Lines 2-3 and Lines 7-8) require $O(N_s \cdot n \cdot \log n)$ work, while merging the local scores (Lines 10-11) requires $O(N_s \cdot n)$ work. The resulting total work is

$$O\big((\ell + N_s \cdot n) \log n + n \cdot \log^2 n + \beta \cdot n \cdot d \cdot \log n\big).$$

With public item ranges assigned to $p$ workers for merging, the time complexity of parallel refinement is

$$O\Big(\big(n + \max_i\{\ell_i\}\big) \cdot \log n + N_s \cdot \lceil n/p \rceil + n \cdot \log^2 n/p + \log^2 n + \beta \cdot n \cdot d \cdot \log n + k \cdot d\Big),$$

For the additional space consumption, assume a linear-space ORAM implementation. Then, the local score ORAMs and readback arrays require $O(N_s \cdot n)$ space, while buffering the selected vectors requires $O(\beta \cdot n \cdot d)$ space. Parallel activation additionally requires $O(N_s \cdot \kappa)$ space for the distance sums and centroid identifier pairs. Including the $O(\ell)$ activation buffers, parallel ObliSC requires $O(\ell + N_s \cdot \kappa + N_s \cdot n + \beta \cdot n \cdot d)$ auxiliary space, excluding database and index storage. It thus trades additional storage and aggregation work for concurrent score accumulation, parallel sorting, and parallel distance computation while preserving full obliviousness.

**Correctness and Obliviousness.** For the correctness of parallel ObliSC, Algorithms 5 and 6 preserve the activated candidates and aggregate the same collision scores as the search base. With the same search budgets, arithmetic, and tie-breaking rules, they therefore return the same answer, yielding the following lemma.

**Lemma 3.** *Algorithms 5 and 6 return the same answer as Algorithms 3 and 4.*

Regarding the obliviousness analysis, we follow the security assumptions in Section III. Parallel ObliSC uses independent ORAM instances and local buffers across subspaces. Its parallel execution uses public task assignments and synchronization points, and scheduling decisions do not depend on private query values or intermediate results. Specifically, local scans have public lengths, ORAMs hide query-dependent addresses, and merging and oblivious sorting follow public access schedules. Together with oblivious local operations and a fixed public amount of work per ORAM access, these properties yield an indistinguishability analysis of the complete parallel execution, establishing Theorem 2.

**Theorem 2.** *The parallel* ObliSC *satisfies full obliviousness.*

## V. Experimental Evaluation

We empirically evaluate ObliSC for approximate nearest neighbor search, covering the experimental setup (Section V-A), overall evaluation (Section V-B), parallel evaluation (Section V-C), and parameter studies (Sections V-D–V-G). The source code is publicly available at https://github.com/guozhong-li/ObliSC.

Table 3: Dataset statistics used in the evaluation.

| Dataset | Domain | Feature type | Cardinality | Dim. |
|---|---|---|---|---|
| Deep1M [30] | Image | Deep/CNN feature | 1M | 256 |
| Gist1M [30] | Image | GIST descriptor | 1M | 960 |
| Sift10M [31] | Image | SIFT descriptor | 10M | 128 |
| Yandex Deep10M [32] | Image | Deep embedding | 10M | 96 |
| Sift100M [31] | Image | SIFT descriptor | 100M | 128 |
| Glove [33] | Text | Word embedding | 1.18M | 100 |

### A. Experimental Setup

**Hardware and Software.** We implemented a prototype of ObliSC in C++ and conducted experiments on a server equipped with 2TB RAM and dual 64-core *AMD EPYC* 7742 processors (2.25GHz), which supports *TEE* via the *AMD SEV* technology. All methods except parallel ObliSC used 100 worker threads, each processing one query at a time.

**Datasets and Queries.** We used six ANN datasets covering image and text retrieval. The five image datasets are Deep1M [30], Gist1M [30], Sift10M [31], Yandex Deep10M [32], and Sift100M [31]; the text dataset is Glove [33]. Table 3 reports the dataset domain, feature type, cardinality, and dimensionality. Following SuCo's experimental setting, we used 100 queries per evaluation, with the same query set across all methods for each dataset. For Deep1M and Glove, we used the released ground truth; for Gist1M, Sift10M, Yandex Deep10M, and Sift100M, we computed exact top-100 ground truth by exhaustive distance evaluation over the entire dataset. All methods use the same ground-truth file for each dataset.

**Baselines.** We evaluate the following baselines and our proposed methods in the single-server setting:

- **SuCo** [4]. The plaintext subspace-collision baseline, without query or access-pattern privacy.
- **PACMANN** [12]. A secure ANN baseline combining graph search with private information retrieval.
- **BiSON** [13]. An ORAM-based secure ANN baseline, evaluated under local loopback and emulated WAN conditions.
- ObliSC**-FO**. Our fully oblivious search base, using fixed-budget candidate activation and refinement.
- ObliSC**-FO-par**. The parallel version of ObliSC-FO using the same oblivious backend.

Regarding the secure ANN baseline PP-ANNS [11], although it protects both the query and the dataset, it fails to preserve access pattern privacy and does not provide open-source code. Hence, we omit a direct comparison.

**Performance Metrics.** We report index construction time (s), average online search time (ms), average recall@50, and persistent index size (GB). Search time covers online query processing, excluding offline index construction; index size excludes transient query workspace. Recall@50 measures the fraction of exact top-50 neighbors returned (Equation 1), averaged over the 100 queries, with $k = 50$.

**Default Parameters.** For SuCo and PACMANN, we follow the parameter settings in their original papers [4], [12]. SuCo's collision ratio $\alpha$ and refinement ratio $\beta$ control candidate activation and re-ranking, respectively. For BiSON, we use

graph degree 128, construction/search list size 125, eight search iterations, and beamwidth 32, with identical search parameters under local and WAN conditions.

Following SuCo, the reference configuration for our shared subspace-collision pipeline is $\alpha = 0.03$, $\beta = 0.003$, $\kappa = 50^2$, $N_s = 8$, and $t = 2$. Here, $N_s$, $\kappa$, and $t$ denote the number of subspaces, IMI cells per subspace, and K-means iterations, respectively. ObliSC-FO and ObliSC-FO-par replace the collision ratio $\alpha$ with the fixed collision parameter $\hat{\alpha}$, whose default value is 100. For ObliSC-FO-par, the default worker count per subspace is $p = 2$.

### B. Overall Evaluation

We first evaluate the overall performance of ObliSC and the baselines on all six datasets, measuring index construction time, online search time, recall, and index size in Table 4.

**Index Time.** SuCo builds the plaintext subspace-collision index, PACMANN constructs graph/PIR artifacts, and ObliSC-FO additionally materializes the fixed-shape public index required by Algorithm 2. Across all datasets, ObliSC-FO incurs a moderate construction overhead over SuCo because it materializes the padded public index and fixed query buffers. Specifically, its construction time is 1.08–1.65× that of SuCo across the six datasets. This overhead is much smaller than PACMANN's graph/PIR construction cost: on Sift100M, ObliSC-FO builds the index in 99.13s, while PACMANN requires 14,880s. The shared FO/FO-par index columns retain the archived FO measurements. FO-par adds no index construction overhead. BiSON's indexing cost is dominated by graph construction and includes one-time ORAM initialization for both local and WAN searches.

**Online Search Time.** The reported FO search time is higher than plaintext SuCo's on every dataset. At the same time, ObliSC-FO remains well below PACMANN's online cost on all evaluated datasets: on Sift100M, ObliSC-FO takes 312.74ms per query, compared with 1,688.6ms for PACMANN. FO-par reports lower search times than FO on all six datasets; on Sift100M, the time decreases to 116.18ms. Across six datasets, BiSON's WAN search time is 21.08–28.61× the local value, without changing recall or persistent index size. Both variants use the same backend and search configuration, ensuring a fair comparison of parallel performance.

**Recall@50.** ObliSC-FO has lower recall than SuCo on Deep1M, Gist1M, Sift10M, and Yandex Deep10M; the largest gap is 6.3 percentage points on Gist1M (0.871 versus 0.934). On Sift100M and Glove, ObliSC-FO instead achieves higher recall: 0.984 versus 0.982, and 0.880 versus 0.816, respectively. This dataset-dependent pattern is consistent with the different activation budgets: ObliSC-FO probes a fixed number of IMI cells, whereas SuCo targets a candidate count, so the number of activated items depends on bucket occupancy. For PACMANN, it gives the lowest recall among the baselines under the recall@50 metric. FO-par matches FO's reported recall on all six datasets. BiSON reaches 0.995 recall on Sift10M but 0.688 on Gist1M, showing dataset-dependent quality under the same search parameter settings.

Table 4: Overall performance on six ANN datasets.

| Data | Method | Index Time (s) | Search Time (ms) | Recall@50 | Index Size (GB) |
|---|---|---|---|---|---|
| Deep1M | SuCo | 2.35 | 2.55 | 0.989 | 0.08 |
| | PACMANN | 136.04 | 1094.98 | 0.802 | 0.53 |
| | BiSON-local | 596.61 | 24.81 | 0.971 | 12.01 |
| | BiSON-WAN | | 668.11 | | |
| | ObliSC-FO | 2.65 | 18.57 | 0.956 | 0.44 |
| | ObliSC-FO-par | | 7.48 | | |
| Gist1M | SuCo | 4.29 | 3.42 | 0.934 | 0.10 |
| | PACMANN | 816.77 | 1155.52 | 0.623 | 1.49 |
| | BiSON-local | 1,468.60 | 52.19 | 0.688 | 35.87 |
| | BiSON-WAN | | 1,228.74 | | |
| | ObliSC-FO | 4.98 | 20.50 | 0.871 | 0.35 |
| | ObliSC-FO-par | | 8.86 | | |
| Sift10M | SuCo | 9.38 | 6.30 | 0.974 | 0.64 |
| | PACMANN | 1004.36 | 1282.71 | 0.793 | 2.09 |
| | BiSON-local | 2,221.36 | 33.20 | 0.995 | 67.71 |
| | BiSON-WAN | | 699.73 | | |
| | ObliSC-FO | 10.89 | 47.36 | 0.957 | 1.26 |
| | ObliSC-FO-par | | 31.59 | | |
| Yandex Deep10M | SuCo | 8.32 | 5.05 | 0.968 | 0.64 |
| | PACMANN | 975.90 | 1254.56 | 0.792 | 1.93 |
| | BiSON-local | 1,867.33 | 28.41 | 0.990 | 57.86 |
| | BiSON-WAN | | 668.69 | | |
| | ObliSC-FO | 9.53 | 50.32 | 0.953 | 1.22 |
| | ObliSC-FO-par | | 43.35 | | |
| Sift100M | SuCo | 60.17 | 83.99 | 0.982 | 6.40 |
| | PACMANN | 14,880 | 1688.60 | 0.564 | 15.94 |
| | BiSON-local | 12,012.43 | 32.45 | 0.938 | 570.25 |
| | BiSON-WAN | | 756.86 | | |
| | ObliSC-FO | 99.13 | 312.74 | 0.984 | 9.62 |
| | ObliSC-FO-par | | 116.18 | | |
| Glove | SuCo | 2.73 | 2.34 | 0.816 | 0.10 |
| | PACMANN | 163.22 | 1091.29 | 0.604 | 0.21 |
| | BiSON-local | 462.52 | 21.61 | 0.921 | 13.23 |
| | BiSON-WAN | | 618.28 | | |
| | ObliSC-FO | 2.95 | 25.63 | 0.880 | 0.17 |
| | ObliSC-FO-par | | 8.42 | | |

**Index Size.** ObliSC-FO's IMIs include the SuCo-side public index with redundant IMI cell elements in each IMI bucket, which are used for padding to ensure full obliviousness. Its reported index size is consistently larger than SuCo's, but smaller than PACMANN's on every benchmark. In particular, on Sift100M, ObliSC-FO's index size is 9.62GB, compared with 6.4GB for SuCo and 15.94GB for PACMANN. BiSON's larger startup footprint includes encrypted graph and vector records, rather than identifier-only index overhead.

### C. Parallel Evaluation

We examined how workers per subspace $p$ affect ObliSC-FO-par's search efficiency by varying $p$ from 1 to 5 on all six datasets, with other parameters unchanged (shown in Table 5). The lowest average batch search time per query occurs at $p = 2$ on every dataset. Increasing $p$ from 1 to 2 reduces the time from 15.77ms to 7.48ms on Deep1M and from 178.43ms to 116.18ms on Sift100M. Across datasets, the speedup over $p = 1$ ranges from 1.17× on Yandex Deep10M to 2.34× on Glove.

Table 5: Effect of $p$ on search time (ms).

| Dataset | $p=1$ | $p=2$ | $p=3$ | $p=4$ | $p=5$ |
|---|---|---|---|---|---|
| Deep1M | 15.77 | **7.48** | 20.61 | 27.21 | 36.02 |
| Gist1M | 18.63 | **8.86** | 20.81 | 25.26 | 32.51 |
| Sift10M | 62.15 | **31.59** | 74.10 | 84.59 | 111.09 |
| Yandex Deep10M | 50.72 | **43.35** | 62.28 | 70.64 | 108.80 |
| Sift100M | 178.43 | **116.18** | 249.40 | 206.42 | 323.29 |
| Glove | 19.67 | **8.42** | 25.08 | 31.39 | 37.49 |

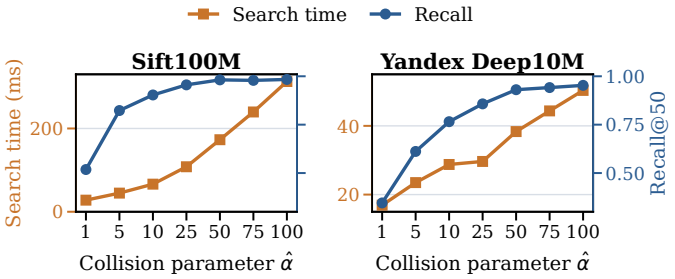



Figure 4: Effect of $\hat{\alpha}$ on search time and recall.

Further increasing $p$ generally worsens performance: at $p=5$, the corresponding times rise to 36.02ms and 323.29ms.

This trend suggests a tradeoff between parallel speedup and execution overhead. Although more workers can accelerate computation, parallel oblivious sorting requires synchronization between dependent network layers, and additional workers may incur greater coordination and memory-contention costs. Beyond $p=2$, these overheads appear to outweigh the gains from further parallelism. The observed performance supports our default choice of $p=2$.

### D. Collision-Parameter Tradeoff

We evaluated how the collision parameter $\hat{\alpha}$ affects ObliSC's recall and search time, since ObliSC's candidate activation (Algorithm 3) accesses exactly $\hat{\alpha}$ IMI cells per subspace. We varied $\hat{\alpha}$ over 1, 5, 10, 25, 50, 75, and 100 on Sift100M and Yandex Deep10M and reported the results in Figure 4.

It can be observed from Figure 4 that ObliSC exhibits the expected quality-cost tradeoff on both datasets: on Sift100M, ObliSC's recall rises from 0.518 at $\hat{\alpha}=1$ to 0.981 at $\hat{\alpha}=50$ and then saturates near 0.984; on Yandex Deep10M, a similar pattern appears, but the recall benefits more gradually: it increases from 0.346 to 0.931 by $\hat{\alpha}=50$ and reaches 0.953 at $\hat{\alpha}=100$. Meanwhile, the search time on both datasets grows as $\hat{\alpha}$ increases. The reason for such trends is that a larger $\hat{\alpha}$ leads to more IMI cell accesses, taking ObliSC more time to activate more candidates.

### E. Query Parameter Sensitivity

We examined how the query parameter $k$ affects ObliSC's search efficiency by varying $k$ over 1, 2, 5, 10, 20, 50, and 100 on Sift100M and Yandex Deep10M, and reported the results in Figure 5.

We can observe from Figure 5 that Recall@$k$ remains high across all $k$: ObliSC's recall on Sift100M stays between 0.983 and 0.99, while the recall on Yandex Deep10M changes

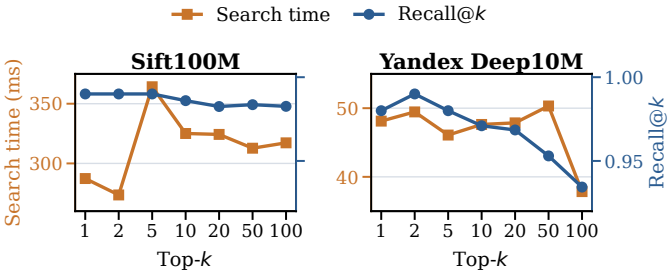



Figure 5: Search-time and recall sensitivity to top-$k$.

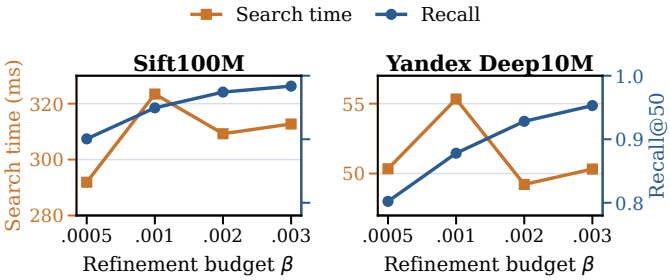



Figure 6: Search-time and recall sensitivity to $\beta$.

smoothly from 0.98 at $k=1$ to 0.934 at $k=100$. However, on both datasets, the recall decreases as $k$ increases. This is because $k$ affects only the final truncation in ObliSC's candidate refinement, making it harder to recover a larger set of exact nearest neighbors from an unchanged set of candidates. Regarding the search efficiency, we can see that ObliSC's search time on Sift100M remains around 300ms for all $k$, with reasonable fluctuation across runs. A similar trend can also be observed for Yandex Deep10M.

### F. Refinement Budget Sensitivity

The refinement budget, namely the re-rank ratio $\beta$, determines that ObliSC's candidate refinement (Algorithm 4) must refine $\beta \cdot n$ candidates. To investigate how $\beta$ affects ObliSC's performance, we varied $\beta$ over 0.0005, 0.001, 0.002, and 0.003 on Sift100M and Yandex Deep10M. The experimental results are reported in Figure 6.

As shown in Figure 6, increasing $\beta$ consistently improves the utility of ObliSC's output result: recall rises from 0.901 to 0.984 on Sift100M and from 0.802 to 0.953 on Yandex Deep10M. However, ObliSC's search time remains within a comparatively narrow band because ObliSC's candidate activation dominates these runs, although a larger $\beta$ takes ObliSC more time for computing more original distances and sorting more elements. Fluctuations of 25ms on a 300ms search time remain within an acceptable range of experimental variance. For Yandex Deep10M, the larger gain in recall when increasing $\beta$ indicates greater benefit from re-ranking additional collided candidates in SC; the search times follow a similar trend to those on Sift100M. Thus, $\beta$ is principally a recall knob once the collided candidates in SC have been fixed, and the default $\beta=0.003$ favors the high-recall end of this tradeoff.

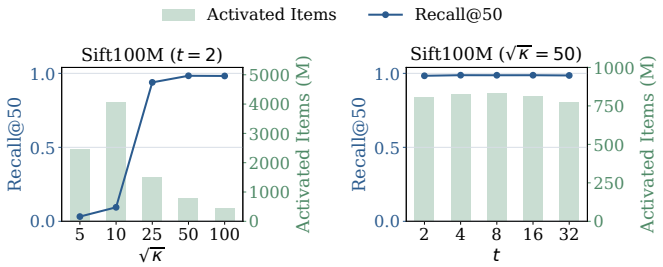



Figure 7: Sensitivity to $\sqrt{\kappa}$ and $t$ on Sift100M.

### G. K-means Parameter Sensitivity

We further studied how the K-means parameters, namely $\kappa$ and $t$, govern IMI bucket balance.

**IMI Granularity $\sqrt{\kappa}$.** The IMI granularity $\sqrt{\kappa}$ is the number of K-means clusters in each half-subspace, giving $\kappa$ cells per IMI. We first varied $\sqrt{\kappa}$ over 5, 10, 25, 50, and 100, and investigated how $\sqrt{\kappa}$ affects the recall and the number of activated items (including padding across subspaces) in ObliSC's candidate activation (Algorithm 3) on Sift100M. As shown by the LHS subfigure of Figure 7, moderate granularity balances recall and activated items: recall rises from 0.032 at $\sqrt{\kappa} = 5$ to 0.984 at $\sqrt{\kappa} = 50$, while finer cells reduce activation. Activated items peak at $\sqrt{\kappa} = 10$ and then decrease. However, at $\sqrt{\kappa} = 100$, fewer items are activated but recall drops slightly. This is because overly fine granularity may cause some true-positive items to be excluded from the activated IMI bucket cell.

**K-means Clustering Iterations $t$.** We varied the K-means clustering iterations $t$ over 2, 4, 8, 16, and 32, and also reported the recall and the number of activated items in the RHS subfigure of Figure 7. Activated items first increase from 804.50M at $t = 2$ to 833.18M at $t = 8$, then decrease to 771.67M at $t = 32$. Recall remains between 0.984 and 0.988, with no monotonic gain from more iterations, supporting $t = 2$ as the default.

## VI. Related Work

**Index-Based ANN Search.** Common ANN indexing and search techniques include product quantization [34], locality-sensitive hashing (LSH) [35], inverted files, trees [36], and graph-based methods such as HNSW [2]. Recently, *subspace collision* (SC) [4] has offered another approach: it aggregates collision information across multiple lower-dimensional subspaces to estimate proximity in the original space. Building on this framework, SuCo [4] combines a lightweight clustering-based index with efficient query-processing strategies.

The efficiency gains of these methods often rely on query-dependent behavior, including bucket and cluster selection, graph traversal, pruning, and candidate refinement. While such adaptivity is natural in plaintext retrieval systems, the resulting access patterns can leak information about a private query vector. Designing ANN search methods that protect query privacy against access-pattern leakage while maintaining high search efficiency therefore remains an important problem.

**Privacy-Preserving ANN Search.** In the single-server setting, Liu et al. [11] studied ANN search with protection for both query and database contents, but their construction does not hide query-dependent access patterns. Li et al. [14] proposed Panther, which combines PIR, secret sharing, garbled circuits, and homomorphic encryption to protect both query and database privacy while hiding query-dependent access patterns against semi-honest adversaries. Das et al. [13] proposed BiSON, which combines DiskANN with a disk-compatible ORAM architecture to protect database contents, queries, and access patterns from an untrusted server in an outsourced setting. Zhou et al. [12] proposed Pacmann, a graph-based ANN search framework that protects query privacy and hides query-dependent graph accesses via Private Information Retrieval (PIR) over a public database. Like Pacmann, ObliSC protects queries over a public vector database, using a trusted execution environment (TEE) for query confidentiality and oblivious search within the TEE to hide access patterns.

In the two-server setting, Servan-Schreiber et al. [9] proposed Preco, a locality-sensitive hashing-based private ANN protocol using distributed point functions for private retrieval from two non-colluding servers, protecting the query while limiting database information disclosed to the client. In the federated setting, Zhang et al. [15] proposed FedKNN for secure kNN search across private data providers. FedKNN combines TEE with differential obliviousness to protect the providers' data against access-pattern leakage.

**Oblivious Search.** Full obliviousness [25] provides strong privacy guarantees by making observable access patterns indistinguishable across inputs of the same public size. Prior work has employed oblivious designs, under different security models and leakage guarantees, for a wide range of query and data-processing tasks, including SQL queries [37]–[42], graph algorithms such as traversal, shortest-path computation, and minimum spanning trees [43]–[45], and graph pattern queries [46]–[50].

## VII. Conclusion

In this paper, we study asymmetric privacy-preserving ANN search over a public vector database, where user queries must remain private from an untrusted cloud server. We propose ObliSC, an efficient search framework whose ANN index, candidate activation, and candidate refinement are jointly designed to protect query privacy against access-pattern leakage. We prove that its search base satisfies the privacy guarantee of full obliviousness. We further develop parallel strategies for candidate activation and refinement while preserving full obliviousness and the search base's results. Experiments on six standard ANN benchmarks demonstrate that ObliSC achieves competitive efficiency and result quality while maintaining a lightweight index compared with the evaluated privacy-preserving approaches. As for future work, we may optimize each of ObliSC's components, implement ObliSC using alternative cryptographic tools rather than a TEE to protect query values and private intermediate values, and extend ObliSC to support additional query workloads.

## AI/LLM USAGE DISCLOSURE

Generative AI tools were used only to polish the writing and assist with code implementation. All technical contributions, including the proposed method, analysis, and experiments, were developed and verified by the authors.

## REFERENCES


[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, 2020.

[2] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.

[3] Y. Peng, B. Choi, T. N. Chan, J. Yang, and J. Xu, "Efficient approximate nearest neighbor search in multi-dimensional databases," *ACM PACMMOD*, vol. 1, no. 1, pp. 54:1–54:27, 2023.

[4] J. Wei, X. Lee, Z. Liao, T. Palpanas, and B. Peng, "Subspace collision: An efficient and accurate framework for high-dimensional approximate nearest neighbor search," *ACM PACMMOD*, vol. 3, no. 1, pp. 79:1–79:29, 2025.

[5] P. A. Norberg, D. R. Horne, and D. A. Horne, "The privacy paradox: Personal information disclosure intentions versus behaviors," *Journal of consumer affairs*, vol. 41, no. 1, pp. 100–126, 2007.

[6] C. Nyst and T. Falchetta, "The right to privacy in the digital age," *Journal of Human Rights Practice*, vol. 9, no. 1, pp. 104–118, 2017.

[7] European Parliament and Council of the European Union, "Regulation (EU) 2016/679 of the European Parliament and of the Council."

[8] Apple and Google, "Exposure notification privacy-preserving analytics (enpa) white paper," Tech. Rep., 2021.

[9] S. Servan-Schreiber, S. Langowski, and S. Devadas, "Private approximate nearest neighbor search with sublinear communication," in *IEEE SP*, 2022, pp. 911–929.

[10] E. Stefanov, M. van Dijk, E. Shi, C. W. Fletcher, L. Ren, X. Yu, and S. Devadas, "Path ORAM: an extremely simple oblivious RAM protocol," in *ACM CCS*, 2013, pp. 299–310.

[11] Y. Liu, Y. Zhang, J. Xie, H. Li, J. X. Yu, and J. Cui, "Privacy-preserving approximate nearest neighbor search on high-dimensional data," in *IEEE ICDE*, 2025, pp. 3017–3029.

[12] M. Zhou, E. Shi, and G. Fanti, "Pacmann: Efficient private approximate nearest neighbor search," in *ICLR*, 2025.

[13] S. Das, R. Ravi, N. Chandran, and D. Gupta, "Bison: Billion-scale oblivious nearest-neighbor search in milliseconds," in *IEEE EuroS&P*, 2026, pp. 160–172.

[14] J. Li, Z. Huang, M. Zhang, C. Hong, J. Liu, T. Wei, and W. Chen, "Panther: Private approximate nearest neighbor search in the single server setting," in *ACM CCS*, 2025, pp. 365–379.

[15] X. Zhang, Q. Wang, C. Xu, Y. Peng, and J. Xu, "Fedknn: Secure federated k-nearest neighbor search," *ACM PACMMOD*, vol. 2, no. 1, pp. V2mod011:1–V2mod011:26, 2024.

[16] J. Wei, Z. Liao, R. Han, Q. Xu, C. Yang, and T. Palpanas, "Taco: Data-adaptive and query-aware subspace collision for high-dimensional approximate nearest neighbor search," *ACM PACMMOD*, vol. 4, no. 3, pp. 241:1–241:28, 2026.

[17] A. Shamir, "How to share a secret," *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.

[18] A. Beimel, "Secret-sharing schemes: A survey," in *Springer IWCC*, vol. 6639, 2011, pp. 11–46.

[19] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *ACM STOC*, 2009, pp. 169–178.

[20] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," *ACM TOCT*, vol. 6, no. 3, pp. 13:1–13:36, 2014.

[21] J. H. Cheon, A. Kim, M. Kim, and Y. S. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Springer ASIACRYPT*, vol. 10624, 2017, pp. 409–437.

[22] I. Corporation, "Intel® trust domain extensions (intel tdx)," White Paper, 2025.

[23] D. Kaplan, J. Powell, and T. Woller, "Amd memory encryption," White Paper, 2021.

[24] N. Corporation, "Ai security with confidential computing," 2023.

[25] O. Goldreich, "Towards a theory of software protection and simulation by oblivious rams," in *ACM STOC*, 1987, pp. 182–194.

[26] "Efficient privacy-preserving high-dimensional approximate nearest neighbor search," https://github.com/guozhong-li/ObliSC/ObliSC-TR.pdf, Tech. Rep., 2026.

[27] J. Wei, Z. Liao, R. Han, Q. Xu, C. Yang, and T. Palpanas, "Taco: Data-adaptive and query-aware subspace collision for high-dimensional approximate nearest neighbor search," *ACM PACMMOD*, vol. 4, no. 3, pp. 241:1–241:28, 2026.

[28] X. Wang, T. H. Chan, and E. Shi, "Circuit ORAM: on tightness of the goldreich-ostrovsky lower bound," in *ACM CCS*, 2015, pp. 850–861.

[29] K. E. Batcher, "Sorting networks and their applications," in *AFIPS*, vol. 32, 1968, pp. 307–314.

[30] The Chinese University of Hong Kong, "GQR: Vector datasets," https://www.cse.cuhk.edu.hk/systems/hash/gqr/datasets.html, accessed: November 9, 2026.

[31] TexMex Research Team, "Datasets for approximate nearest neighbor search," http://corpus-texmex.irisa.fr/.

[32] D. Baranchuk and A. Babenko, "Benchmarks for billion-scale similarity search," https://research.yandex.com/datasets/biganns, 2021, accessed: November 9, 2026.

[33] ANN-Benchmarks, "GloVe-100-angular dataset," https://ann-benchmarks.com/, accessed: November 9, 2026.

[34] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE TPAMI*, vol. 33, no. 1, pp. 117–128, 2011.

[35] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *ACM STOC*, 1998, pp. 604–613.

[36] G. Li, B. Choi, R. Zuo, S. S. Bhowmick, and J. Xu, "lesax index: A learned SAX representation index for time series similarity search," in *IEEE ICDE*, 2025, pp. 1995–2008.

[37] A. Arasu and R. Kaushik, "Oblivious query processing," in *ICDT*, 2014, pp. 26–37.

[38] J. Bater, G. Elliott, C. Eggen, S. Goel, A. N. Kho, and J. Rogers, "SMCQL: secure querying for federated databases," *PVLDB*, vol. 10, no. 6, pp. 673–684, 2017.

[39] S. Eskandarian and M. Zaharia, "Oblidb: Oblivious query processing for secure databases," *PVLDB*, vol. 13, no. 2, pp. 169–183, 2019.

[40] S. Krastnikov, F. Kerschbaum, and D. Stebila, "Efficient oblivious database joins," *PVLDB*, vol. 13, no. 11, pp. 2132–2145, 2020.

[41] A. Mavrogiannakis, X. Wang, I. Demertzis, D. Papadopoulos, and M. N. Garofalakis, "OBLIVIATOR: oblivious parallel joins and other operators in shared memory environments," in *USENIX Security*, 2025, pp. 8521–8540.

[42] Z. Wu, W. Dong, and X. Hu, "Differentially oblivious multi-way join," *ACM PACMMOD*, vol. 4, no. 3, pp. 162:1–162:26, 2026.

[43] M. Blanton, A. Steele, and M. Aliasgari, "Data-oblivious graph algorithms for secure computation and outsourcing," in *ACM CCS*, 2013, pp. 207–218.

[44] K. Nayak, X. S. Wang, S. Ioannidis, U. Weinsberg, N. Taft, and E. Shi, "Graphsc: Parallel secure computation made easy," in *IEEE SP*, 2015, pp. 377–394.

[45] J. G. Chamani, I. Demertzis, D. Papadopoulos, C. Papamanthou, and R. Jalili, "Graphos: Towards oblivious graph processing," *PVLDB*, vol. 16, no. 13, pp. 4324–4338, 2023.

[46] Z. Fan, B. Choi, J. Xu, and S. S. Bhowmick, "Asymmetric structure-preserving subgraph queries for large graphs," in *IEEE ICDE*, 2015, pp. 339–350.

[47] L. Xu, J. Jiang, B. Choi, J. Xu, and S. S. Bhowmick, "Privacy preserving strong simulation queries on large graphs," in *IEEE ICDE*, 2021, pp. 1500–1511.

[48] L. Xu, B. Choi, Y. Peng, J. Xu, and S. S. Bhowmick, "A framework for privacy preserving localized graph pattern query processing," *ACM PACMMOD*, vol. 1, no. 2, pp. 129:1–129:27, 2023.

[49] S. Wang, Y. Zheng, X. Jia, and C. Wang, "egrass: An encrypted attributed subgraph matching system with malicious security," *IEEE TIFS*, vol. 19, pp. 5999–6014, 2024.

[50] L. Xu, H. Hu, Y. Zheng, J. Jiang, Q. Ye, B. Choi, and J. Xu, "Pmatch: A secure framework for querying private localized graph patterns," *IEEE TKDE*, vol. 38, no. 10, pp. 6926–6941, 2026.

## APPENDIX A
## PROOFS

Throughout the proofs, we use the privacy assumptions stated in Section III.

**Lemma 2.** `TopkRefine` *satisfies full obliviousness.*

*Proof.* Consider two executions with the same public inputs but different queries and activated identifiers. By Lemma 1, each $R_i$ has public size $\hat{\alpha} \cdot Max[i]$. Algorithm 4 scans these arrays in fixed order, performing one ORAM-protected read-modify-write per identifier, including virtual identifiers through the dummy slot. It then obliviously sorts the $n$ real-item scores, retrieves exactly $\beta \cdot n$ vectors through ORAM, and computes their distances using fixed local access schedules. The distance sort is oblivious, and the final $k$ vectors are retrieved through ORAM and written to public output positions while their contents remain hidden from SP. Thus, local schedules and request counts are query-independent. Adaptive ORAM security hides the private addresses throughout these operations, yielding computationally indistinguishable traces and establishing Equation (2). □

**Theorem 1.** ObliSC *satisfies full obliviousness.*

*Proof.* Consider two executions of ObliSC with the same public database, index, and parameters but different queries. Algorithm 2 constructs the index independently of the query. Algorithm 1 then performs initialization and query projection using access schedules determined by public sizes and the index partition. It invokes candidate activation and refinement in a fixed order. By Lemmas 1 and 2, both stages have query-independent local schedules and ORAM request counts. Adaptive ORAM security hides the private addresses across the complete sequence, including requests sharing ORAM state, while private intermediate values and results remain hidden from SP. Thus, sequentially composing these stages yields computationally indistinguishable traces and establishes Equation (2). □

**Lemma 3.** *Algorithms 5 and 6 return the same answer as Algorithms 3 and 4.*

*Proof.* Consider the sequential and parallel executions with the same index, query, budgets, arithmetic, and tie-breaking rules. Algorithms 3 and 5 compute and rank the same centroid-pair distances and retrieve the same first $\hat{\alpha}$ padded cells. Thus, each $R_i$ contains the same identifiers with the same multiplicities. Algorithm 6 counts each real identifier's occurrences locally and sums these counts, reproducing the global scores in Algorithm 4. Virtual identifiers update only dummy slots, and counters accommodate the public maximum without overflow. Both procedures therefore select the same $\beta \cdot n$ identifiers. The distance computations use the same arithmetic order, and both sorts use the same tie-breaking rules. Consequently, the final top-$k$ answers are identical. □

**Theorem 2.** *Parallel* ObliSC *satisfies full obliviousness.*

*Proof.* Consider two parallel executions with the same public inputs and worker configuration but different queries. Under the scheduling conditions in Section IV, Algorithm 5 assigns distance computations, sorting comparisons, and padded-cell reads to workers independently of the query. Each output $R_i$ has public size $\ell_i$. Algorithm 6 initializes $n + 1$ score slots per subspace, performs exactly $\ell_i$ updates including dummy updates, and reads back $n$ real scores. The ORAM instances have independent randomness and states. Applying adaptive ORAM security to each instance in turn yields indistinguishable joint traces, even though their requests may be correlated through the query.

Merging and sorting follow public access schedules. Candidate retrieval and final result retrieval issue exactly $\beta \cdot n$ and $k$ requests through the shared database ORAM, respectively. Distance computations use public buffer positions and worker assignments, and output writes visit fixed positions while their contents remain hidden from SP. Oblivious local operations and public per-access work bounds prevent additional value-dependent leakage. Under the conditions in Section IV, scheduling and synchronization introduce no private dependence beyond the protected traces. Thus, the complete observable executions, including thread identities and synchronization events, are computationally indistinguishable, establishing Equation (2) up to the negligible error of the secure primitives. □